

Enriching Pre-Training Using Fuzzy Logic

Vansh Gupta*, Vandana Bharti*, Abhinav Kumar, Anshul Sharma, and Sanjay Kumar Singh

Abstract—Graph representation learning advances graph machine learning by encoding structural and relational information into feature vectors. This study introduces a fuzzy logic-based pre-processing layer that enhances node representations by adding semantic diversity and contextual understanding. The layer models uncertainty and captures abstract semantic characteristics in graph data, addressing the limitations of conventional methods that depend solely on structural attributes. By applying defuzzification, the layer refines embeddings, improving their robustness and effectiveness for a wide range of downstream tasks. To test its robustness, we introduce Gaussian noise ranging from 2% to 10% into the datasets, simulating real-world data imperfections. We evaluate the proposed layer on GraphMamba architecture, using DeepWalk and Node2Vec as baseline node feature generation algorithms. The results show consistent improvements in accuracy, F1 scores, precision, and recall across different noise levels. Our findings demonstrate the layer’s ability to preserve high representational quality, speed up convergence, and handle noisy representations effectively.

Index Terms—Graph Representation Learning, Fuzzy Logic, GraphMamba, Robust Learning.

I. INTRODUCTION

Graph data are central to domains like social media, biology, and physical sciences, with examples including Twitter interactions, Reddit discussions, and protein-protein networks. Extracting and reconstructing these structures is key to graph representation and generative learning. Techniques such as deep embeddings, graph convolutions, and message-passing networks have shown strong performance in applications like chemical modeling, 3D vision, and recommendation systems.

However, most methods rely on structure, ignoring semantic uncertainty. Fuzzy logic addresses this by modeling vagueness and enabling reasoning under uncertainty. Models like FCGNN [1], FuzzyDGL [2], and Deep Fuzzy Clustering [3] incorporate fuzzy representations to improve flexibility and robustness. In contrast, stochastic models like DeepWalk [4], Node2Vec [5], and Struc2Vec [6] focus on structural

*Vansh Gupta and Vandana Bharti are joint first authors and contributed equally to this work.

Corresponding Authors: Abhinav Kumar and Anshul Sharma

Vansh Gupta is with the Department of Physics, Indian Institute of Technology BHU, Varanasi, India. (e-mail: vansh.gupta.phy21@iitbhu.ac.in).

Vandana Bharti is with the Department of Computer Science and Engineering, Indian Institute of Technology Dharwad, India. (e-mail: vandana@iitdh.ac.in).

Abhinav Kumar is with the School of Artificial Intelligence and Data Engineering, Indian Institute of Technology Ropar, Rupnagar, India. (e-mail: abhinavk@iitrpr.ac.in).

Anshul Sharma is with the Department of Information Technology, ABV-Indian Institute of Information Technology & Management Gwalior, India. (e-mail: anshul@iiitm.ac.in).

Sanjay Kumar Singh is with the Department of Computer Science and Engineering, Indian Institute of Technology BHU, Varanasi, India. (sks.cse@iitbhu.ac.in).

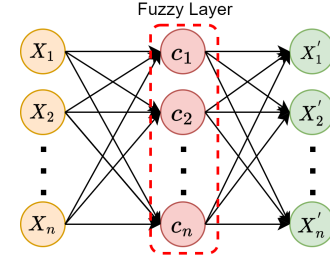


Figure 1. Generation of new embedding set by introducing a fuzzy layer.

patterns. GNNs such as GCNs [7], GraphSAGE [8], and GATs [9] integrate features and topology but lack mechanisms for semantic enrichment.

This paper addresses the need for a synergistic framework that combines semantic richness and computational efficiency. We propose a fuzzy logic-based pre-processing layer, illustrated in Fig. 1, which enhances node representations generated by existing algorithms. The proposed layer enriches input node representations with abstracted semantic information and outputs defuzzified representations. By leveraging fuzzy logic, this layer captures the ambiguity and contextual nuances of graph data, facilitating improved performance in downstream tasks. Our approach is versatile in the sense that it can be integrated with any node representation learning algorithm whether it is for static graphs or dynamic graphs. Dynnode2vec [10] is an adaptation of node2vec for dynamics graphs. We also have deep learning based techniques like SiGNN [11] which incorporate graph neural networks for node representation learning. We can effortlessly integrate our layer with a dynamic graph node representation learning algorithm such as the above two without any modifications.

We integrate the fuzzy pre-processing layer with GraphMamba [12] architecture and evaluate its performance on two datasets: BIOGRID-human [13] and Arxiv CA-GrQc [14] with and without noise. The experimental results indicate significant enhancements in accuracy, F1 scores, precision and recall values, underscoring the layer’s effectiveness in advancing graph representation learning.

This paper presents the following key contributions:

- We propose a novel fuzzy logic-based pre-processing layer that integrates seamlessly with existing graph representation learning algorithms. This layer enhances node embeddings by combining semantic enrichment with structural representation, addressing limitations of conventional approaches that rely solely on structural features.
- The proposed layer achieves a theoretical time complexity

of $O(N^2 \times K) + O(K \times d \times N^2) + O(d \times N)$, where N denotes the number of nodes, K represents the number of clusters, and d is the embedding vector dimension.

- Experimental results demonstrate the effectiveness of the fuzzy layer on two datasets— CA-GrQc and BIOGRID-Human. Improvements of around 2% are observed in the accuracy and F1-scores which establishes the legitimacy of the proposed layer.
- Even at higher noise levels (e.g., 10%), the fuzzy layer preserves robust performance, underscoring its ability to handle noisy graph embeddings effectively.

The remainder of the paper is organized as follows: Section II reviews the foundational concepts relevant to understand the proposed approach. Section III details the architecture and key components of the proposed layer. Section IV describes the experimental setup used to evaluate the effectiveness of the method. Finally, Section V concludes the paper. Together, these sections provide the context, methodological insights, and evaluation framework for the proposed work.

II. BACKGROUND

A. Fuzzy Logic Systems

Fuzzy logic systems provide a mathematical framework to handle uncertainty and ambiguity by allowing partial membership of elements in sets. These systems extend classical logic and set theory, making them ideal for real-world problems characterized by imprecision.

1) *Fuzzy Sets*: Fuzzy sets are an extension of traditional sets, where the membership of an element to a set is not limited to a binary value of 0 or 1 but can take any value between 0 and 1, allowing for a more nuanced representation of variables. A fuzzy set A in a universe of discourse X is defined by its membership function $\mu_A(x)$:

$$\mu_A(x) : X \rightarrow [0, 1] \quad (1)$$

where $\mu_A(x)$ represents the degree of membership of x in the set A . For example, $\mu_A(x) = 0$ implies no membership, $\mu_A(x) = 1$ implies full membership, and intermediate values indicate partial membership.

2) *Membership Functions*: Membership functions map input values to their degrees of membership in a fuzzy set. Commonly used functions include:

- Gaussian:

$$\mu_A(x) = \exp\left(-\frac{(x-c)^2}{2\sigma^2}\right) \quad (2)$$

where c is the mean and σ^2 is the variance.

- Triangular:

$$\mu_A(x) = \begin{cases} 0 & x \leq a \text{ or } x \geq c \\ \frac{x-a}{b-a} & a < x \leq b \\ \frac{c-x}{c-b} & b < x < c \end{cases} \quad (3)$$

where a, b, c define the triangle's vertices.

3) *Fuzzy Inference System*: It consists of the rules that govern the decision-making process based on the input variables and their corresponding membership functions. These rules are typically expressed in the form of IF-THEN statements, where the IF part represents the antecedent or condition based on input variables, and the THEN part represents the consequent or action to be taken based on those conditions.

4) *Rule Base*: The rule base is a collection of IF-THEN rules that defines the behaviour of the fuzzy logic system.

5) *Defuzzification*: The defuzzification process is used to convert the fuzzy output of the system into a crisp or numerical value that can be easily understood and used in practical applications. There are currently two major Fuzzy Systems: the Takagi Sugeno-Kang fuzzy system (TSK-FS) [15], [16] and the Mamdani-Larsen fuzzy system (ML-FS) [17], [18]. To describe a TSK-FS in terms of its rule base:

$$\begin{aligned} \text{IF } x_1 \text{ is } A_1^k \wedge x_2 \text{ is } A_2^k \wedge \dots \wedge x_d \text{ is } A_d^k, \\ \text{THEN } f(x)^k = p_0^k + p_1^k x_1 + p_2^k x_2 + \dots + p_d^k x_d \end{aligned} \quad (4)$$

where $k = 1, 2, \dots, N$, k is the sequence number of the fuzzy rules, and N is the total number of fuzzy rules in the rule base; $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$ is the d -dimensional input vector; A_i^k ($i = 1, 2, \dots, d$) denotes the fuzzy subset associated with the input variable x_i for the k -th rule in the antecedents; $\wedge$ is the fuzzy conjunction operator; $f^k(\mathbf{x})$ is the output of the k -th rule; and p_i^k ($i = 1, 2, \dots, d$) are the real-valued parameters of the rule consequent.

B. Graph Fuzzy Systems (GFS)

Graph Fuzzy Systems extend fuzzy logic principles to graph-structured data, enabling uncertainty management in graph analytics. These systems adapt the components of classical fuzzy systems to graphs.

1) *Rule Base*: The graph fuzzy-inference rule base of GFS extends the classical TSK-FS rule base and is defined as follows [19].

$$\text{IF } \text{graph} \text{ is } GF^k, \text{ THEN } f^k(\text{graph}) = GCPU^k(\text{graph}) \quad (5)$$

In the GFS defined above:

- a)  represents the input graph. This will be interchangeably used with G .
- b) GF^k () denotes the graph fuzzy set corresponding to the antecedent of the k -th fuzzy rule, which denotes a fuzzy set of graphs, and () denotes the prototype graph of the fuzzy set, that is, the graph data sample that can best represent the fuzzy set;
- c) $GCPU^k$ () denotes the graph consequent processing unit (GCPU) corresponding to the k -th fuzzy rule consequent. In general, the rule can be stated as

$$\text{IF } G \text{ is } GF^k, \text{ THEN } f^k(G) = GCPU^k(G) \quad (6)$$

2) *Membership Functions*: The membership function $\text{GraphSim}(G, G_{pt})$ measures the similarity between the input graph G and prototype graph G_{pt} . For instance:

$$\text{GraphSim}(G, G_{pt}) = \frac{\sum_{v \in V} \mu_v(G, G_{pt})}{|V|} \quad (7)$$

where $\mu_v(G, G_{pt})$ quantifies the node-level similarity, and $|V|$ is the number of nodes.

3) *Antecedents and Their Generation*: Each antecedent in GFS corresponds to a fuzzy graph set derived through clustering. Methods such as k-means [20], fuzzy c-means (FCM) [21], or hierarchical clustering [21] can partition graph data into meaningful clusters. For graph data, clustering requires adapting these techniques to account for graph-specific features like connectivity and edge attributes.

4) *Graph Consequent Processing Unit (GCPU)*: The GCPU processes the consequent of each rule by utilizing Graph Neural Networks (GNNs) such as GCN, GAT, or GraphSAGE. Each GCPU applies:

$$f_k(G) = \text{Trans}^k(\text{Agg}^k(G)) \quad (8)$$

where Agg^k aggregates features from neighboring nodes, and Trans^k transforms these features into outputs. Among all the K rules, each rule can use different GCPUs (e.g., different GNNs) to perform different Agg and Trans operations. It can be seen from [22] that feature aggregation and transformation of each rule is the core of the decision-making of the GFS rule Consequent.

C. Decision-Making in GFS

The decision-making process in GFS utilizes fuzzy inference principles to combine the contributions of multiple fuzzy rules. Each rule evaluates the membership of the input graph G in a graph fuzzy set GF^k and computes a corresponding output using the Graph Consequent Processing Unit (GCPU). The decision function integrates these rule-specific outputs into a final decision by aggregating them with membership-based weights.

The decision function of GFS is defined as:

$$f_{GFS}(G) = \sum_{k=1}^K \mu_{GF^k}(G) \cdot GCPU^k(G) \quad (9)$$

where $\mu_{GF^k}(G)$ is the membership value of graph G in the k -th fuzzy graph set GF^k , calculated as:

$$\mu_{GF^k}(G) = \text{GraphSim}(G, G_{pt}^k) \quad (10)$$

and G_{pt}^k is the prototype graph of GF^k . $GCPU^k(G)$ is the output of the Graph Consequent Processing Unit for the k -th rule.

The proposed fuzzy pre-processing layer enhances graph representation learning by integrating fuzzy membership and defuzzification, enriching node embeddings with semantic and contextual information. This method addresses the limitations of purely structural approaches by bridging graph topology with semantic content, resulting in more robust embeddings for tasks such as classification and link prediction.

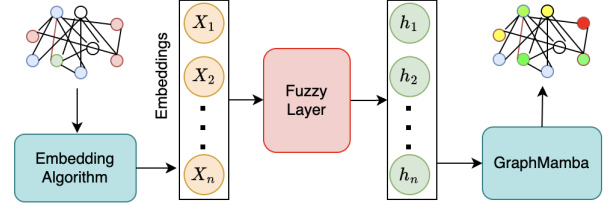


Figure 2. Training process workflow. The graph dataset is first processed using a node embedding algorithm (e.g., Node2Vec or DeepWalk) to generate initial node embeddings. These embeddings are then refined by the proposed fuzzy pre-processing layer to improve their semantic and contextual quality. The enhanced embeddings are subsequently input into the Graph architecture to perform downstream task.

III. METHODOLOGY

The workflow of the proposed framework, as shown in Fig. 2, integrates fuzzy logic principles into graph representation learning. This framework enhances node embeddings through a fuzzy pre-processing layer and then utilizes these enriched embeddings within the GraphMamba architecture to perform link prediction. The three main steps in the methodology are as follows:

- 1) Node embeddings are generated using algorithms such as Node2Vec and DeepWalk which encode structural and relational graph information into low-dimensional vector representations.
- 2) The proposed fuzzy pre-processing layer refines these embeddings by incorporating semantic diversity and contextual nuances.
- 3) The enriched embeddings are input into the Graph architectures.

The fuzzy pre-processing layer comprises two key components: K-Means Node Clustering and the Fuzzy Logic System, which further splits into Antecedent and Consequent phases. Their details are elaborated below.

A. K-Means Node Clustering

Traditional K-means clustering assigns data points to clusters based on their Euclidean distance to cluster centroids. However, this method is unsuitable for graph-based node embeddings, as these embeddings lie in a high-dimensional latent space. To address this, we adapt the K-means algorithm to treat each node embedding as a multidimensional vector and compute distances across these dimensions.

Let $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$ denote the set of node embeddings, where $\mathbf{x}_i \in \mathbb{R}^d$. The algorithm minimizes the objective function:

$$J = \sum_{k=1}^K \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \mu_k\|^2 \quad (11)$$

where K is the number of clusters, C_k is the set of points in the k -th cluster, and μ_k is the centroid of the k -th cluster.

Algorithm 1: K-Means Node Clustering

Input: Number of clusters K , node embeddings $\mathbf{X}$, embedding dimension d

Output: Cluster assignments $\mathbf{C}$ and centroids μ

```

1 Initialize: Randomly initialize centroids
    $\mu_1, \dots, \mu_K \in \mathbb{R}^d$ 
2 repeat
3   foreach  $\mathbf{x}_i \in \mathbf{X}$  do
4     Assign  $\mathbf{x}_i$  to the cluster  $C_k$  with the nearest
       centroid  $\mu_k$ 
5   Update each centroid:
       
$$\mu_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$$

6 until Centroids converge or max iterations reached
7 return Cluster assignments  $\mathbf{C}$  and centroids  $\mu$ 

```

Algorithm 1 describes the clustering process. Each node embedding is assigned to the cluster with the nearest centroid, and centroids are iteratively updated until convergence.

B. Fuzzy Logic System

The fuzzy logic system processes node embeddings through Antecedent and Consequent components, as described below.

1) *Antecedent*: The antecedent transforms node embeddings into membership values for fuzzy sets. Using the Gaussian membership function, the membership value $h_z(x_i)$ for node i in dimension z is given by:

$$h_z(x_i) = \exp\left(-\frac{(x_i - \mu_z)^2}{2\sigma_z^2}\right) \quad (12)$$

where μ_z is the cluster centroid for dimension z , and σ_z^2 is the variance of the cluster. Algorithm 2 computes these membership values. After the membership values corresponding to all the nodes along each dimension of node feature have been calculated for all the fuzzy sets, we take their average to get the mean membership value.

2) *Consequent*: The consequent aggregates the membership values by averaging them to generate defuzzified embeddings. By calculating μ_i^{mean} , we get a membership value that is representative of all the clusters. Defuzzification is performed by considering d (dimension of node feature vector) distributions, corresponding to each dimension, and calculating the mean and variance. This works on the idea that we have a representative membership value and the gaussian distribution. The sole objective is to determine the input value corresponding to a given membership value. Applying the following formula

$$y_i = \sqrt{-\log(\mu_i^{mean})} * \sigma_k^2 + \mu_k \quad (13)$$

where y_i is the defuzzified value and μ_i^{mean} is the mean membership vector for i -th node. σ_k^2 and μ_k are the variance and mean for k -th distribution out of the d distributions described above. Algorithm 3 describes the process.

Algorithm 2: Antecedent Computation

Input: Node embeddings $\mathbf{X}$, centroids

$\mu = \{\mu_1, \mu_2, \dots, \mu_K\}$, embedding dimension d

Output: Membership values matrix $\mathbf{H} \in \mathbb{R}^{N \times d}$

```

1 Initialize: Empty membership matrix  $\mathbf{H} \in \mathbb{R}^{N \times d}$ 
2 Compute mean of the centroids:

```

$$\mu_m = \frac{1}{K} \sum_{k=1}^K \mu_k$$

3 Compute variance of the centroids:

$$\sigma^2 = \frac{1}{K} \sum_{k=1}^K (\mu_k - \mu_m)^2$$

4 **for** $i = 1$ to N **do**

5 **for** $z = 1$ to d **do**

6 Compute membership value:

$$h_z(x_i) = \exp\left(-\frac{(x_i^z - \mu_m^z)^2}{2\sigma_z^2}\right)$$

Store $h_z(x_i)$ in $\mathbf{H}[i][z]$

7 **return** $\mathbf{H}$

Algorithm 3: Consequent Computation

Input: Membership matrix $\mathbf{H}$, centroids μ

Output: Defuzzified embeddings $\mathbf{Y}$

1 **Initialize:** Empty output matrix $\mathbf{Y}$

2 **foreach** $\mathbf{x}_i \in \mathbf{X}$ **do**

3 Compute $y_i = \sqrt{-\log(\mu_i^{mean})} * \sigma_k^2 + \mu_k$

4 Append y_i to $\mathbf{Y}$

5 **return** $\mathbf{Y}$

C. Integration with GraphMamba

While supplying these modified enhanced embeddings to GraphMamba, we experiment with different noise level in the original embeddings. If the original embeddings, generated by Node2Vec, or DeepWalk, are $\mathbf{X}$, we introduce error κ ranging from 2%-10%.

$$\mathbf{X} + \kappa \rightarrow \mathbf{X}_\kappa \quad (14)$$

$$FuzzyLayer(\mathbf{X}_\kappa) \rightarrow \mathbf{X}_\kappa^{Fuzzy} \quad (15)$$

Both $\mathbf{X}_\kappa$ and $\mathbf{X}_\kappa^{Fuzzy}$ are then passed to the GraphMamba architecture.

IV. EXPERIMENTS

A. Experimental Setup

We evaluated the proposed framework by integrating the fuzzy pre-processing layer with GraphMamba [12] to assess its performance on noise-induced embeddings. The model was tested on the link prediction task to validate its effectiveness.

Table I
EVALUATION METRICS FOR DEEPWALK AND NODE2VEC ON CA-GrQc AND BIOGRID-HUMAN DATASETS

Algo	E(%)	Type	CA-GrQc				BIOGRID-Human			
			Accuracy(%)	F1 (%)	Precision(%)	Recall(%)	Accuracy(%)	F1(%)	Precision(%)	Recall(%)
DeepWalk	0%	NonFuzzy	98.03±0.003	98.07±0.003	96.27±0.007	100.00±0.000	95.74±0.015	95.57±0.016	99.50±0.001	92.77±0.041
		Fuzzy	97.46±0.005	97.52±0.005	95.17±0.010	100.00±0.000	96.60±0.023	96.50±0.026	98.33±0.008	94.97±0.055
	2%	NonFuzzy	98.01±0.003	98.03±0.003	97.03±0.007	99.06±0.001	96.22±0.014	96.11±0.015	99.49±0.002	94.29±0.040
		Fuzzy	97.46±0.005	97.50±0.005	95.85±0.008	99.49±0.001	97.39±0.008	97.48±0.008	97.83±0.013	98.50±0.006
	5%	NonFuzzy	97.31±0.001	97.32±0.001	96.96±0.006	97.82±0.003	91.33±0.055	90.28±0.068	98.91±0.060	84.97±0.014
		Fuzzy	97.24±0.008	97.30±0.008	96.50±0.010	99.53±0.002	97.42±0.005	97.44±0.004	96.72±0.015	98.84±0.007
	8%	NonFuzzy	96.9±0.0037	96.98±0.007	97.66±0.006	97.54±0.003	95.21±0.010	95.21±0.011	99.23±0.002	95.54±0.031
		Fuzzy	97.01±0.011	97.08±0.011	97.37±0.022	99.37±0.004	97.10±0.009	97.10±0.010	97.29±0.011	98.87±0.007
	10%	NonFuzzy	97.52±0.006	97.52±0.006	97.42±0.002	98.49±0.010	95.40±0.010	95.34±0.011	97.79±0.022	95.10±0.037
		Fuzzy	97.32±0.003	97.37±0.003	97.48±0.007	99.92±0.007	95.44±0.030	95.31±0.034	97.27±0.013	95.65±0.085
Node2Vec	0%	NonFuzzy	97.68±0.002	97.73±0.002	95.57±0.005	100.00±0.000	98.28±0.001	98.29±0.001	99.32±0.001	98.59±0.008
		Fuzzy	96.91±0.005	97.01±0.004	94.19±0.09	100.00±0.000	97.86±0.006	97.85±0.007	98.36±0.006	97.56±0.011
	2%	NonFuzzy	97.57±0.003	97.59±0.003	96.85±0.005	98.85±0.001	95.85±0.019	95.71±0.021	99.36±0.001	93.62±0.041
		Fuzzy	97.56±0.006	97.61±0.005	95.94±0.014	99.61±0.003	97.23±0.005	97.22±0.005	97.44±0.011	98.54±0.017
	5%	NonFuzzy	97.33±0.003	97.33±0.003	97.37±0.007	97.62±0.006	96.25±0.006	96.20±0.006	99.46±0.002	95.86±0.023
		Fuzzy	97.61±0.005	97.66±0.005	96.77±0.010	99.70±0.003	97.34±0.003	97.37±0.003	97.33±0.008	98.45±0.008
	8%	NonFuzzy	97.18±0.004	97.18±0.004	97.56±0.006	97.61±0.010	96.53±0.003	96.53±0.003	98.93±0.003	96.65±0.008
		Fuzzy	97.53±0.009	97.59±0.009	96.81±0.007	99.66±0.001	97.06±0.004	97.10±0.004	97.00±0.011	98.34±0.014
	10%	NonFuzzy	97.09±0.005	97.11±0.05	97.60±0.005	97.86±0.014	92.83±0.052	92.25±0.034	99.02±0.004	88.82±0.083
		Fuzzy	97.30±0.006	97.33±0.006	97.30±0.007	99.67±0.000	97.24±0.003	97.27±0.003	98.07±0.008	98.71±0.010

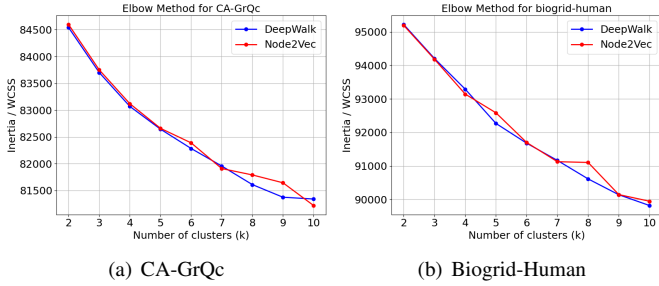


Figure 3. Elbow Method Visualization for Optimal Cluster Selection in Datasets

1) *Datasets*: We conducted experiments on the following datasets for link prediction:

Arxiv CA-GrQc: This dataset represents the collaboration network in General Relativity and Quantum Cosmology, sourced from e-print arXiv submissions. It contains 5242 nodes and 14496 edges [14].

BIOGRID-human: This dataset is a gene interaction network where nodes represent genes and edges represent interactions between them. It consists of 9527 nodes and 62364 edges [13].

2) *Embedding Methods*: To generate the initial embeddings, we employed the following algorithms:

DeepWalk [4]: This method treats random walks on the graph as sentences, applying language modeling techniques to generate embeddings.

Node2Vec [5]: Node2Vec optimizes a trade-off between local and global graph structure by introducing biased random walks.

3) *Training Details*: The dataset is divided using a 90:10 train:test split. For the link prediction task, negative edges—non-existent in the original graph are sampled in equal

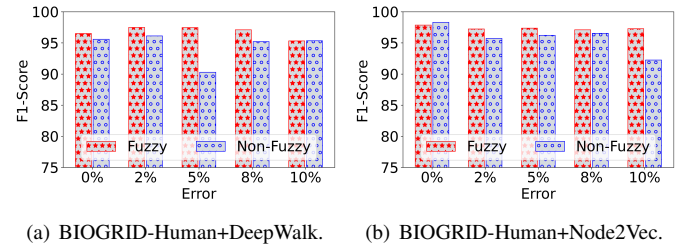


Figure 4. Model performance for Biogrid-Human test dataset. Fuzzy pre-training enhances F1-Score in many cases.

number to test edges. The experiments employ GraphMamba with two GraphGPS layers, following the configuration in [3], using 64 channels and 4D positional encoding. The model is trained with Gaussian noise levels ranging from 2% to 10% applied to the embeddings. To ensure robustness, results are averaged over five independent runs.

Cluster selection is based on elbow analysis of the node embeddings. Five clusters are optimal for the CA-GrQc dataset, showing clear elbows in both embeddings in Fig.3(a). In the Biogrid-human dataset, only DeepWalk reveals an elbow at five clusters, shown in Fig.3(b). To ensure consistency, five clusters are used for both datasets.

B. Results and Discussion

The impact of the fuzzy pre-processing layer is illustrated in Table. I, which compares model performance with and without fuzzy pre-training. A visual comparison of Accuracy and F1-scores is shown in Fig.4 and Fig.5.

1) *Fuzzy Pre-training Analysis*: Table I presents results of noise-induced training using GraphMamba. Across all settings, the fuzzy layer consistently improves performance and en-

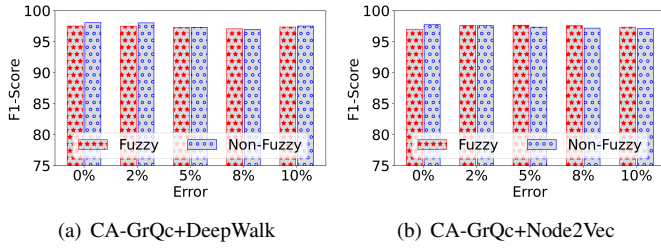


Figure 5. Model performance for CA-GrQC test dataset. Fuzzy pre-training enhances F1-Score in many cases.

hances robustness, even at noise levels up to 10%. Gains of 2–2.5% across metrics indicate improved embedding quality. While effectiveness varies by dataset and noise level, the layer shows notable impact—particularly at 8–10% noise on the smaller CA-GrQC dataset and from as early as 5% noise on the larger Biogrid-Human dataset, where it yields a 5% accuracy boost. Fig. 4 and Fig. 5 further highlight that larger datasets benefit more, likely due to greater latent structure and enrichment potential under noise.

2) *Trade-offs and Limitations:* The fuzzy pre-processing layer performs well when the number of clusters and dataset size are balanced. A higher number of clusters allows for better granularity in large datasets, but over-clustering can dilute relevant information, increasing computational cost without improving performance.

3) *Complexity Analysis:* The fuzzy layer operates with a time complexity of:

$$O(N^2 \times K) + O(K \times d \times N^2) + O(d \times N), \quad (16)$$

where N is the number of nodes, K is the number of clusters, and d is the embedding dimension. This complexity is manageable for medium-sized datasets and demonstrates the feasibility of integrating fuzzy logic into graph representation learning.

V. CONCLUSION

This paper demonstrates that incorporating fuzzy pretraining significantly enhances the quality of embedding vectors by enriching them with semantic and contextual information. The results highlight the versatility of the proposed pre-processing layer and its effectiveness in improving graph embeddings by capturing and leveraging contextual relationships. This establishes the layer as a valuable addition to pre-training process. Its superior performance on noisy embeddings underscores the robustness of fuzzy logic as a promising approach for improving representation learning in graph-based tasks. By refining the algorithm to reduce complexity and boost performance, the approach could scale effectively to larger datasets. For dynamic graphs, we can apply a dynamic node representation learning algorithm and subsequently enrich the resulting embeddings through our fuzzy layer, yielding more robust and informative representations.

REFERENCES

- [1] C.-Y. Zhang, Y.-N. Lin, C. P. Chen, H.-Y. Yao, H.-C. Cai, and W.-P. Fang, “Fuzzy representation learning on graph,” *IEEE Transactions on Fuzzy Systems*, vol. 31, no. 10, pp. 3358–3370, 2023.
- [2] H.-Y. Yao, Y.-L. Yu, C.-Y. Zhang, Y.-N. Lin, and S.-J. Li, “Fuzzy representation learning on dynamic graphs,” *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, no. 2, pp. 878–890, 2024.
- [3] Q. Feng, L. Chen, C. P. Chen, and L. Guo, “Deep fuzzy clustering—a representation learning approach,” *IEEE Transactions on Fuzzy Systems*, vol. 28, no. 7, pp. 1420–1433, 2020.
- [4] B. Perozzi, R. Al-Rfou, and S. Skiena, “Deepwalk: Online learning of social representations,” in *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2014, pp. 701–710.
- [5] A. Grover and J. Leskovec, “node2vec: Scalable feature learning for networks,” in *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, 2016, pp. 855–864.
- [6] L. F. Ribeiro, P. H. Saverese, and D. R. Figueiredo, “struc2vec: Learning node representations from structural identity,” in *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, 2017, pp. 385–394.
- [7] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *arXiv preprint arXiv:1609.02907*, 2016. [Online]. Available: <https://arxiv.org/abs/1609.02907>
- [8] W. Hamilton, Z. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” *Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS 2017)*, vol. 30, pp. 1025–1035, 2017.
- [9] P. Casanova, A. R. P. Lio, and Y. Bengio, “Graph attention networks,” *ICLR. Petar Velickovic Guillem Cucurull Arantxa Casanova Adriana Romero Pietro Liò and Yoshua Bengio*, 2018.
- [10] S. Mahdavi, S. Khoshraftar, and A. An, “dynnode2vec: Scalable dynamic network embedding,” 2018. [Online]. Available: <http://arxiv.org/abs/1812.02356>
- [11] D. Chen, S. Zheng, M. Xu, Z. Zhu, and Y. Zhao, “Signn: A spike-induced graph neural network for dynamic graph representation learning,” *Pattern Recognition*, vol. 158, p. 111026, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0031320324007775>
- [12] C. Wang, O. Tsepa, J. Ma, and B. Wang, “Graph-mamba: Towards long-range graph sequence modeling with selective state spaces,” *arXiv preprint arXiv:2402.00789*, 2024.
- [13] R. Rossi and N. Ahmed, “The network data repository with interactive graph analytics and visualization,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 29, no. 1, 2015.
- [14] J. Leskovec, J. Kleinberg, and C. Faloutsos, “Graph evolution: Densification and shrinking diameters,” *ACM transactions on Knowledge Discovery from Data (TKDD)*, vol. 1, no. 1, pp. 2–es, 2007.
- [15] T. Takagi and M. Sugeno, “Fuzzy identification of systems and its applications to modeling and control,” *IEEE transactions on systems, man, and cybernetics*, no. 1, pp. 116–132, 1985.
- [16] M. Sugeno and G. Kang, “Structure identification of fuzzy model,” *Fuzzy sets and systems*, vol. 28, no. 1, pp. 15–33, 1988.
- [17] Mamdani, “Application of fuzzy logic to approximate reasoning using linguistic synthesis,” *IEEE transactions on computers*, vol. 100, no. 12, pp. 1182–1191, 1977.
- [18] P. M. Larsen, “Industrial applications of fuzzy logic control,” *International Journal of Man-Machine Studies*, vol. 12, no. 1, pp. 3–10, 1980.
- [19] F. Hu, Z. Deng, Z. Xie, K.-S. Choi, and S. Wang, “Graph fuzzy system: Concepts, models and algorithms,” *arXiv preprint arXiv:2210.16730*, 2022. [Online]. Available: <https://arxiv.org/abs/2210.16730>
- [20] J. MacQueen *et al.*, “Some methods for classification and analysis of multivariate observations,” in *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, vol. 1, no. 14. Oakland, CA, USA, 1967, pp. 281–297.
- [21] J. C. Bezdek, R. Ehrlich, and W. Full, “Fcm: The fuzzy c-means clustering algorithm,” *Computers & geosciences*, vol. 10, no. 2-3, pp. 191–203, 1984.
- [22] W. L. Hamilton, R. Ying, and J. Leskovec, “Representation learning on graphs: Methods and applications,” *arXiv preprint arXiv:1709.05584*, 2017. [Online]. Available: <https://arxiv.org/abs/1709.05584>